Tech spec: Smart Filters

Smart Filters

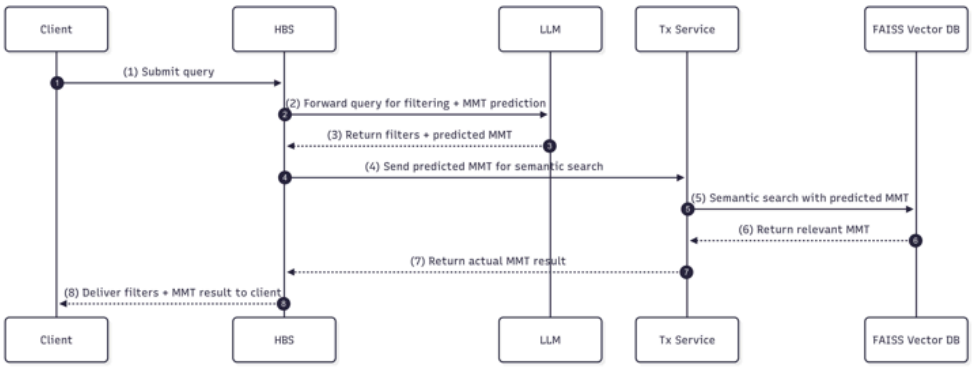
Problem statement

Users struggle to discover and apply relevant filters on the Listing Page View (LPV), especially under the "More Filters" section. This leads to confusion and inefficiency in refining search results, as many users are unaware of the available filtering options.

Solution

To implement an intelligent filter extraction mechanism. The aim is to allow users to search using natural language queries and retrieve refined, structured filters that power the Algolia-based search on the client.

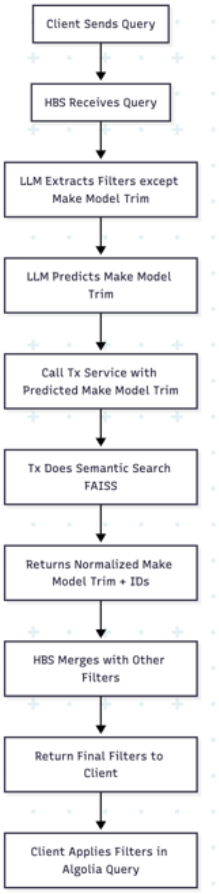
Sequence diagram



Services Involved

- **HBS (Horizontal Buyer Server)**: Acts as the main interface for the client and is responsible for query understanding and filter extraction.
- **Tx (Taxonomy Service)**: Responsible for normalizing predicted make/model/trim using vector-based semantic matching.

Flow Diagram



Key Components

- **LLM (Large Language Model)**: Customized per category to understand domain-specific queries.
- **Vector DB**: Enables fuzzy matching and semantic normalization of `make`, `model`, and `trim`.
- **Algolia**: Final search engine used on the client side.

Smart Filter Extraction(LLM)

- HBS will utilize GPT-4o mini (via OpenAI), Langchain, and OutputParser for natural language understanding and structured output generation.
- This LLM extracts structured filter data based on all known properties and values for that category (e.g., color, year, body type), **excluding** `make`, `model`, and `trim`.

- The LLM **predicts** `make`, `model`, and `trim` from the user's query without using a predefined list.
- **LLM prompt creation is dynamic** based on the selected category (e.g., 'used cars' or 'rental cars'). This means the LLM smart filter extractor knows exactly which properties and values to extract according to that category's filter configuration.

```
SYSTEM_PROMPT = '''
            You are a smart filter extractor for a car listing platform.
            The user's query may be in Arabic or English.

            Your task:
            - Extract only the filters mentioned in the user's natural language query.
            - Always return the filters as a clean JSON object using English keys and
English values.

            Supported fields (in English keys):
            - make
            - model
            - trim (e.g., SE, XLE, Limited, GT, Platinum, G63)
            - year_min (calculated from the current year)
            - year_max (calculated from the current year)
            - price_min
            - price_max
            - transmission_type (must be one of: automatic, manual)
            - mileage_max
            - mileage_min
            - condition
            - regional_specs (must be one of: GCC specs, American specs,
            Japanese specs, Chinese specs, Canadian specs, Other specs)
            - origin (must be one of: German, Japanese, Chinese, American, Other)
            - sort (must be one of: date_desc, date_asc, price_desc, price_asc)
            - body_type (e.g., SUV, Sedan, Pickup, Hatchback, Coupe, Convertible, Van,
Wagon)
            - extras (must be a list and only include values from the following):
                - Air Conditioning
                - Alarm/Anti-Theft System
                - AM/FM Radio
                - Aux Audio In
                - Bluetooth System
                - Body Kit
                - Brush Guard
                - Cassette Player
                - CD Player
                - Climate Control
                - Cooled Seats
                - DVD Player
                - Fog Lights
                - Heat
                - Heated Seats
                - Keyless Entry
                - Keyless Start
                - Leather Seats
                - Moonroof
                - Navigation System
                - Off-Road Kit
                - Off-Road Tyres
                - Parking Sensors
                - Performance Tyres
                - Power Locks
                - Power Mirrors
                - Power Seats
                - Power Sunroof
```

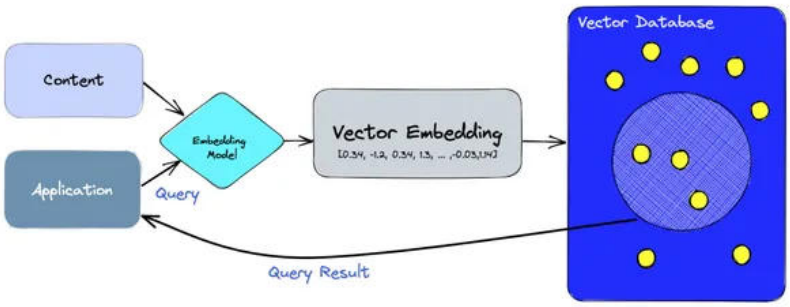
```
                - Power Windows
                - Premium Lights
                - Premium Paint
                - Premium Sound System
                - Premium Wheels/Rims
                - Racing Seats
                - Rear View Camera
                - Roof Rack
                - Satellite Radio
                - Spoiler
                - Sunroof
                - VHS Player
                - Winch
            - doors (must be one of: 2, 3, 4, 5+)
            - interior_color (must be one of: Beige, Black,
            Blue, Brown, Green, Grey, Orange, Red, Tan, White, Yellow, Other Color)
            - exterior_color (must be one of: Black, Blue, Brown,
            Burgundy, Gold, Grey, Orange, Green, Purple, Red, Silver, Beige,
            Tan, Teal, White, Yellow, Other Color)
            - seating_capacity (must be one of: 2, 3, 4, 5, 6, 7, 8, 8+)
            - fuel_type (must be one of: petrol, diesel, hybrid, electric)
            - engine_capacity (must be one of: '0 - 499cc', '500 - 999cc',
            '1000 - 1499cc', '1500 - 1999cc', '2000 - 2499cc', '2500 - 2999cc',
            ', 3000 - 3499cc', '3500 - 3999cc', '4000+ cc', 'Unknown')
            - horsepower (must be one of: '0 - 99 HP', '100 - 199 HP',
            '200 - 299 HP', '300 - 399 HP', '400 - 499 HP', '500 - 599 HP',
            '600 - 699 HP', '700 - 799 HP', '800 - 899 HP', '900+ HP', 'Unknown')
            - no_of_cylinders (must be one of: 3, 4, 5, 6, 8, 10, 12, Unknown)
            - warranty (must be one of: yes, no, Does not apply)
            - steering_side (must be one of: left hand, right hand)
            - city (must be one of: Abu Dhabi, Dubai, Sharjah, Ajman, Ras Al Khaimah,
            Fujairah, Umm Al Quwain, Al Ain, All Cities)

            Rules:
            - User may write in Arabic, but always return keys and values in English.
            - Map "sort by latest" (or Arabic equivalent) → "date_desc"
            - Map "sort by cheapest" (or Arabic equivalent) → "price_asc"
            - transmission_type must be "automatic" or "manual" only — do not guess
            - Do not guess or hallucinate values. Extract only what is clearly mentioned.

            Smart model-to-make inference:
            - If user says "كامري" (Camry) or "تشارجر" (Charger),
            extract both model and known make (Toyota, Dodge).

            Smart trim inference:
            - Recognize trim codes (e.g., "G63") and extract make/model/trim if clearly
associated.

            Output format:
            Return a clean JSON object with English keys and values.
        '''
```

How a Vector Database Works

1. **Ingestion**
   - Store each known make/model/trim as its precomputed embedding + metadata (ID, label).
2. **Indexing**
   - OpenSearch builds FAISS HNSW index with byte vectors for ultra-fast similarity lookup.
3. **Querying**

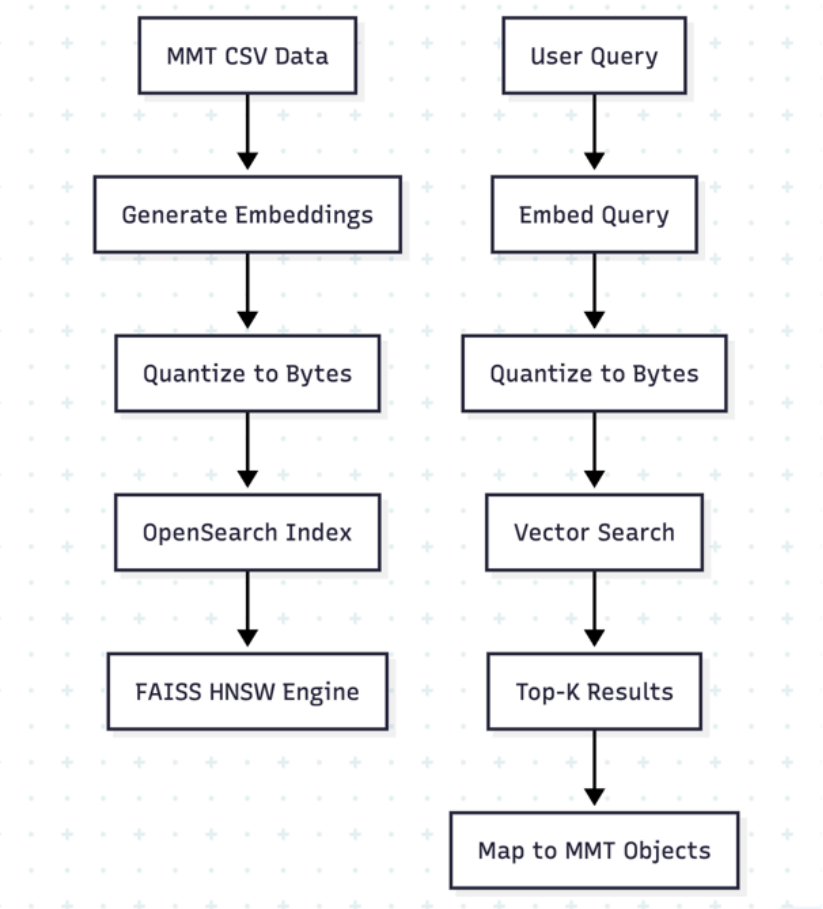
- Convert user query to byte vector and perform ANN search in OpenSearch.

4. **Normalization**
   - Map OpenSearch results back to canonical make/model/trim objects.



Used case: Introducing byte vector support for Faiss in the OpenSearch vector engine

How do we get our mmt from string



⚡ **Performance Benefits with Byte Vectors**

Based on the OpenSearch FAISS byte vector research:

Metric	Float32 Vectors	**Byte Vectors**	**Improvement**
Memory Usage	100%	**28%**	**72% reduction**
Indexing Speed	Baseline	**2x-107.84% faster**	**Massive speedup**
Storage Size	4 bytes/dim	**1 byte/dim**	**4x smaller**
Quality Loss	0%	**<8.8%**	**Minimal impact**

📈 Complete Performance Landscape

Here's the full comparison across three approaches:

1. Local FAISS (In-Memory) vs 2. OpenSearch Float32 vs 3. OpenSearch Byte

Metric	**Local FAISS**	**OpenSearch Float32**	**OpenSearch Byte**
Deployment	Single server	Cloud distributed	Cloud distributed
Memory Usage	200-500MB local RAM	Cloud memory	**72% less cloud memory**
Indexing Speed	Local CPU speed	Cloud distributed	**2x-107.84% faster than float32**
Query Latency	50-100ms	100-200ms	**80-150ms**
Scalability	Limited to one machine	Unlimited horizontal	Unlimited horizontal
Availability	Single point of failure	High availability	High availability
Storage	Local disk (~50-100MB)	Cloud storage	**4x smaller cloud storage**
Maintenance	Manual updates	Managed service	Managed service

Costing

Daily traffic : Current used-cars LPV receives ~200k requests/day. Based on this assuming smart filters gets similar traffic.

Component	unit cost	cost/day
OpenAI GPT 4o-mini LLM	$0.0002/request > Input: `1,000 tokens` = `$0.00015` > Output: `100 tokens` = `$0.0006`	$40
OpenSearch Service	indexing : $.25/hour querying : $.25/hour	$12
Embeddings using all-MiniLM-L6-v2 (sentence-transformers)	free	$0
Total		**$52**

MMT Matching API

For all mmts from red env:

- **Makes**: ~533 text variants (213 × 2.5 avg variants)
- **Models**: ~6,034 text variants (1,724 × 2.5 + 1,724 make+model combinations)
- **Trims**: ~14,000 text variants (6,843 × 2 + selective full combinations)

Current response time with local setup is 200ms
Payload sample:

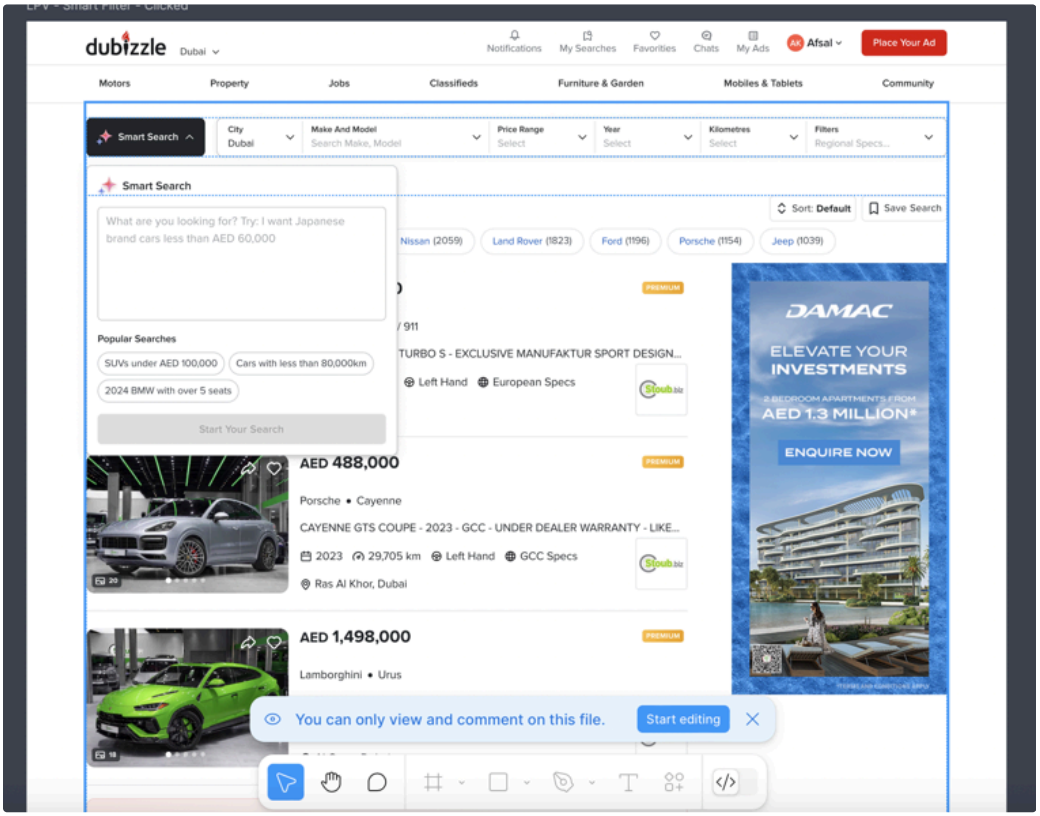
```
{
    "query": "Mercedes-Benz G-Class G63AMG",
    "top_k": 3,
    "min_confidence": 0.7
}
```

Response sample:


```
{
    "query": "Mercedes-Benz G-Class G63AMG",
    "matches": [
        {
            "confidence": 0.903,
            "matched_text": "mercedes-benz g-class g 63 amg",
            "hierarchy": "make:mercedes-benz:1459 model:g-class:1476 trim:g-63-amg:225",
            "make": {
                "id": 1459,
                "name": "Mercedes-Benz",
                "slug": "mercedes-benz"
            },
            "model": {
                "id": 1476,
                "name": "G-Class",
                "slug": "g-class"
            },
            "trim": {
                "id": 225,
                "name": "G 63 AMG"
            }
        },
        {
            "confidence": 0.896,
            "matched_text": "mercedes-benz g-class",
            "hierarchy": "make:mercedes-benz:1459 model:g-class:1476 trim:no:0",
            "make": {
                "id": 1459,
                "name": "Mercedes-Benz",
                "slug": "mercedes-benz"
            },
            "model": {
                "id": 1476,
                "name": "G-Class",
                "slug": "g-class"
            },
            "trim": null
        },
        {
            "confidence": 0.863,
            "matched_text": "mercedes-benz g-class g 65 amg",
            "hierarchy": "make:mercedes-benz:1459 model:g-class:1476 trim:g-65-amg:230",
            "make": {
                "id": 1459,
                "name": "Mercedes-Benz",
                "slug": "mercedes-benz"
            },
            "model": {
                "id": 1476,
                "name": "G-Class",
                "slug": "g-class"
            },
            "trim": {
                "id": 230,
                "name": "G 65 AMG"
            }
        }
    ],
    "total_matches": 3,
    "processing_time_ms": 457
}
```

Designs

https://www.figma.com/design/99K2ArafgeeZD11SZHtF46/Smart-Filters?node-id=11022-1839&t=mzdlmNes784tYeom-0 Connect your Figma account



Feedbacks/Questions

1. Co-locate both the LLM and the Vector Database within the same service (preferably in either Tx or HBS) to reduce latency and improve system efficiency.
2. Let the LLM handle filter prediction and perform semantic search to retrieve relevant data directly from the vector database. This eliminates the need to include large amounts of data in the prompt, reducing token usage and overall cost.
3. The current 200k /definition api requests are hitting HBS , there are quite a few which are cached so check the numbers in nginx.
4. Evaluate costs if using quantized models (llama3.2) on container using lambda.
5. How do you measure the success of this feature ? eg: right filters applied? ; did it even apply filters for some cases?
6. Will there be rate limiting in place against bots/ random user queries?